本章路線

這章要回答的核心問題是：哪些運算需求與侷限推動作業系統功能演進，這些歷史設計如何轉化成今日抽象？

前置知識。 OS 導論、process/memory/file abstraction、computer architecture history。

- 用需求與限制比較歷史系統
- 辨識功能傳承與版本差異
- 避免把歷史產品直接等同當代系統

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

功能遷移不是原實作原封不動搬家

本章以歷史系統觀察「限制→機制→新增契約」，不將章節次序當成嚴格時間線。batch sequencing減少人工等待，spooling把裝置生產／消費解耦，time sharing加入互動與CPU回收，virtual memory把資訊身分與實體駐留分離。

原限制	機制	新增而非自動解決的問題
人工換job慢	resident monitor	如何辨識job終止與保護monitor
讀卡／列印慢	offline I/O、spool	queue、空間、錯誤／重試
單人占機久	time sharing	timer、隔離、response與公平
快memory昂貴	paging／automatic backing	fault重啟、mapping、replacement

Multics、UNIX及後續較小平台出現相似的多使用者、階層檔案與共享機制，能支持概念遷移的討論；若要斷言某段code直接衍生，仍須更具體的source／設計記錄。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較系統先固定四個維度

同一「process」可能是靜態worker、動態建立的process或Mach task內的thread；同一「paging」可能只做relocation，也可能包含demand loading。

1. 執行代理是否動態建立？一個proxy對應一個job，還是多job重用worker？
2. memory address的單位是byte或word？page多大，table為何存在？
3. 服務跨越的是軟體layer、privilege、address space，或獨立機器？
4. fault／timeout／message buffer歸誰？何時可釋放與重試？

這四項不同時，不能因名詞相同就把現代API或效能保證套回歷史案例。本章給定數值的演示模型會明列其規則，不把它們當成當年實機量測。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從手動操作到resident monitor：減掉哪一段等待

早期job經compile、assemble、load、run等步驟，operator換媒體／觀察終止造成CPU空等。batch先把相似需求聚合，monitor再依control record自動選下一步；controller/driver仍處理實際裝置細節。

固定3jobs順序F,C,F，CPU各2分鐘；每次切compiler種類成本3分鐘，初次設定也算。照順序設定F/C/F共9，CPU6，總15；改F,F,C只設定F/C共6，總12。此改善依job允許重排，且未計等待回應與相依性。

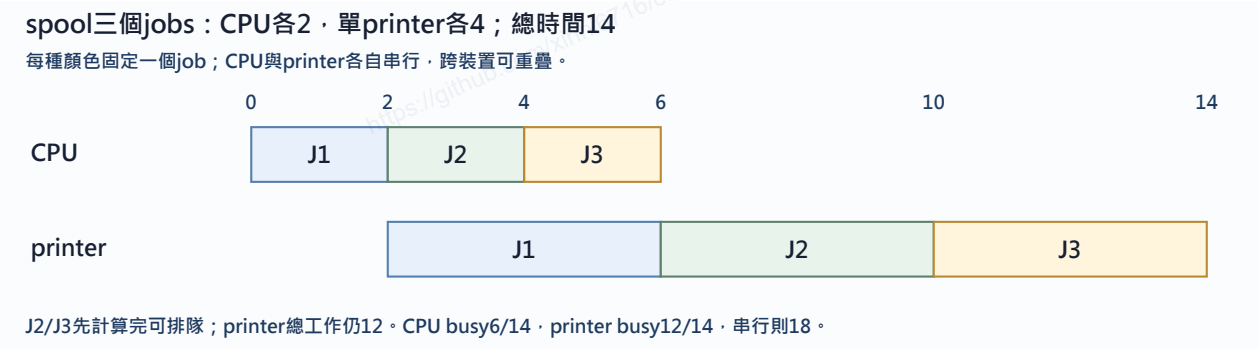
monitor須留在受保護狀態；user job若能覆寫它或直接halt全機，自動sequencing就不可信。control card與data的邊界也需要語法／長度／escape規則，不能單靠資料「通常不含那個字元」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

offline、buffering、spooling與multiprogramming分開

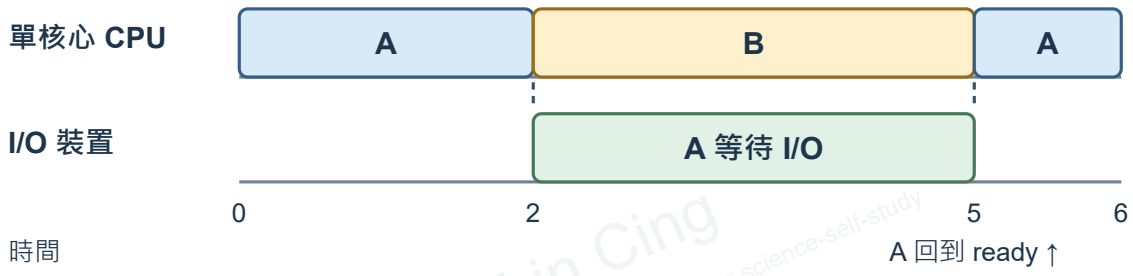
- offline I/O：別的設備先把卡片轉到tape，再帶去主機；省主機等待，但搬移／集滿媒體增加單joblatency。
- buffering：暫存資料以銜接生產者／消費者，容量不足仍會backpressure。
- disk spooling：獨立保存jobs的輸入／輸出與queue metadata，使裝置與不同job的計算可重疊；不能把同一tape想成兩端同時隨機存取。
- multiprogramming：OS讓一job等待時執行另一ready job；需要隔離與排程，spool本身不等同所有工作都已在RAM並行。

給3jobs、每jobCPU2與print4、CPU/單printer可重疊：CPU0–2 J1、2–4 J2、4–6 J3；print2–6 J1、6–10 J2、10–14 J3。總14、CPU6/14、printer12/14；完全串行CPU＋print需18。新增spool不會把printer12縮成6。



一顆 CPU 如何在等待時做別的事

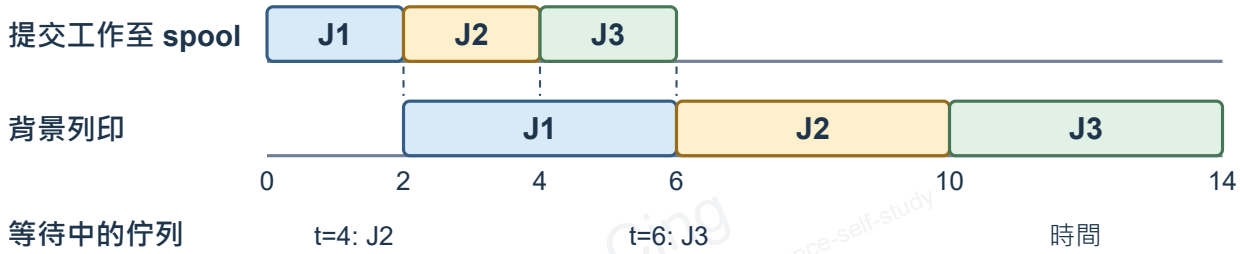
假設 A、B 同時就緒，忽略切換成本；A 執行 2 單位後發出耗時 3 單位的 I/O，B 執行至結束，A 再執行 1 單位。



時間 2–5，A 等待裝置，B 使用 CPU；CPU 和裝置可同時忙，但單核心不會同時執行 A、B 的指令。時間 5 A 回到 ready 並執行。若無其他 ready process，CPU 仍可能閒置。

Spool：送入佇列與實際印完是兩個時點

單 CPU 按序處理 J1–J3，各需 2 單位；提交至容量足夠的 spool。印表機按 FIFO 每份印 4 單位，列印與 CPU 可重疊，忽略其他成本。



時間 4：J2 已提交、等待列印，J1 仍在列印。時間 6：J1 印完、J2 開始，J3 等待；時間 14 全部完成。**提交成功不代表印表完成。** 佇列滿或裝置故障時，應依介面規定處理；buffering 與 spooling 依用途區分。

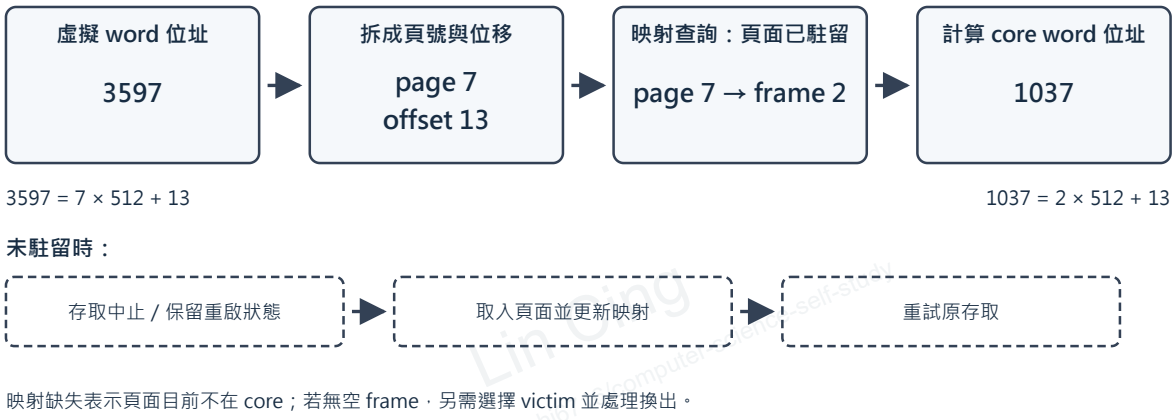
Atlas one-level store：保留資訊位址，自動換駐留位置

Atlas把快core與drum結合成程式看到的一層store；512-word pages與associative mapping，使不在core的block可由系統載入後繼續。原書所述16Kiwords core可分32frames；word不是byte，不把16Kiwords誤稱16KiB。

以縮小的4frames示意，不改512word/page：virtual word address=7×512+13=3597。若page7在frame2，core word address=2×512+13=1037；若不在core，先保存可重啟狀態、取入空frame並更新mapping，才能重試原存取。

one-level抽象不表示每次都像core一樣快，也不讓drum直接當CPU可任意讀的cache-freeRAM。需安排換出、保留空frame與進行中transfer，不能把仍被使用的frame交給另一page。

Atlas：從 word 位址走到 core frame



映射缺失表示頁面目前不在 core；若無空 frame，另需選擇 victim 並處理換出。

3597 = 7 × 512 + 13；若 page 7 映射 frame 2，位址為 1037。缺頁時先取得 frame；無空 frame 才選 victim，修改過的資料必要時先寫回。

Atlas learning replacement：三個分支都要有

Kilburn等1962原論文以t表示距最近使用的時間、T表示上一段inactive間隔；採樣時間依執行指令數，不是普通牆鐘時間。選擇依序：①任一t>T+1；②否則在t≠0者中選最大T−t；③若全t=0，選最大T。tie須另定，本例用表序。

案例	pages的(t,T)	分支／victim
A	(6,3),(2,5),(0,9)	①首page，6>4
B	(2,4),(1,6),(0,8)	②忽略t0；scores2/5，選第二
C	(0,2),(0,5),(0,3)	③全部剛用，選T5的第二

第二分支把過去間隔當下次使用預測；它不是知道future的OPT。若預測最晚的page下一步反而被讀，就會有額外fault。保留第三分支，才能對全部t=0的合法狀態選出victim。原論文。

XDS-940：paging可只做relocation，不必demand paging

依原書歷史配置，process virtual space16Kiwords、physical64Kiwords、page2Kiwords：每process有8virtual pages，physical有32frames。page table用registers保存，允許process頁分散擺放；必要時以整體image swapping增減在RAM的process。

例P的virtual page3→frame11，word offset17：VA=3×2048+17=6161，PA=11×2048+17=22545。這是word address，不乘成bytes；平台word位寬不在本例中額外假設。

可重定位不代表某頁能合法缺席後由demand fault補入。XDS-940本節的paging用於relocation；Atlas本節則自動按需取頁。兩者都有page table，不能據此推成相同memory政策。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

共享reentrant code與新增的控制邊界

兩個process各8pages，完全獨立需16frames。若其中2pages是可安全共用的唯讀reentrant code，則2shared+6private×2=14frames。兩個virtual映射指向同一physicalcode，private可寫資料仍分開。

reentrant不代表所有資料天然安全：若code偷偷修改同一全域buffer，唯讀codepages也不能讓該buffer自動成每process副本。需要區分指令頁與可變狀態。

XDS-930改成940時加入user/monitor、特權I/O/halt與受控system call，使資源操作可由OS中介。動態subprocess與共享memory另外帶來生命期、同步和終止清理；多個保護特性必須一起維持。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

THE分層依賴：上層只在下層已建立的抽象上推理

Dijkstra的THE是batch multiprogramming設計，以明確同步的sequential processes合作，不是以人機terminal互動為目標。

level	責任／對上隱藏的細節
0	processor allocation、timer；上層用sequential-process抽象
1	segment controller／automatic backing；上層用資訊segment身分
2	console message interpreter；上層不用分配實體console
3	I/O buffering／logical communication units
4	user programs
5	operator（不是其軟體實作的level）

這是抽象與依賴順序，不表示每layer都有獨立address space或每次call都有trap。segment是資訊單位、core/drum page是存放位置，兩者身分分開。EWD196。

layer契約的歸納與破壞方式

令第i層在假設下層契約C0…C(i−1)成立時，保證Ci且不破壞下層隱藏狀態。base C0由硬體／processor控制維持；歸納可推到每層。這不等於「分層就自動正確」，每個Ci及干擾範圍仍須證明。

例console level2的message文字放在segment，可使用level1取回，不必全永久留core。若level1轉而呼叫level2才能完成自己的基本取頁，會形成依賴cycle，原先自底向上證明不再適用。

同步也需不依執行速度：一個產生者尚未產生的segment，消費者不能先讀；printer同一時刻不能被兩個不相容owner使用。等待可調整進度，不可把「通常比較慢」當作正確性條件。



RC4000 nucleus把政策留給上面的process

RC4000的目標是提供可組裝不同系統的nucleus，而不是把單一batch／time-sharingpolicy全寫死。它提供process控制、保護與message primitives；上層process可表達不同服務。

原書message固定8words、共用buffer pool；每process有待收message queue。send-message／wait-message及send-answer／wait-answer組成request/reply；primitive原子性只涵蓋其內部狀態更新，不讓整個server操作自動變transaction。

裝置也透過process式介面表示：driver把register／interrupt轉成message。這是統一操作模型，並非硬體真的成為能執行任意usercode的普通process。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2個buffers的ownership守恆

本例固定pool={b0,b1}，一個request/reply完成前保留其buffer；不是逐指令模擬原機。

步	free	server queue／處理中	awaiting client reply
初始	b0,b1	空	空
C1 send req	b1	b0 queued	空
C2 send req	空	b0,b1 queued	空
S receive b0後answer	空	b1 queued	b0
C1 receive answer並歸還	b0	b1 queued	空

每個buffer恰屬一狀態；free+queued+processing+reply=2。第三request遇free=0要等／依API錯誤處理，不得重複配出b0。若S等C1的額外訊息、C1又等S回答，仍可能deadlock；小kernel與atomic send不會單獨解循環等待。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CTSS將短互動與長計算區分對待

CTSS 在 MIT 示範互動式 time sharing，並保留既有 batch 程式的執行環境。user image 在有限 memory 與 backing store 間移動，因此過短 quantum 可能使搬移成本占比過大。「compatible」不代表所有後來的 API 均二進位相容；7090/7094 的記憶體單位是 36-bit word，不可直接換成 KiB。

指數 feedback 將用完整個時間片仍未完成的工作降級，並給較低層較長時間片；I/O 完成後是否提升、aging 與初始層級需另訂。

自行定義的比較模型：單 CPU、切換成本暫不計，L 在 t=0 到達且需 11 單位，短工作 S 在 t=1 到達且需 1 單位。所有新工作進 level 0，$q_0=1$，較高層優先；L 在 [0,1) 用完時間片後降至 level 1。S 在 [1,2) 執行，response time 為 0；若採非搶先 FCFS，L 持續到 t=11，S 的 response time 為 10。這只是該負載與規則的比較，不是對所有互動工作的保證。

固定指數feedback模型的完整burst軌跡

固定 $q_i=2^i$、i 從 0 開始：只有一個 CPU burst，長度 11，從 level 0 開始；用完時間片且尚未完成才降一層，無競爭工作及 promotion。

dispatch	level	時間片	實際用量	累計 CPU	剩餘
1	0	1	1	1	10
2	1	2	2	3	8
3	2	4	4	7	4
4	3	8	4	11	0

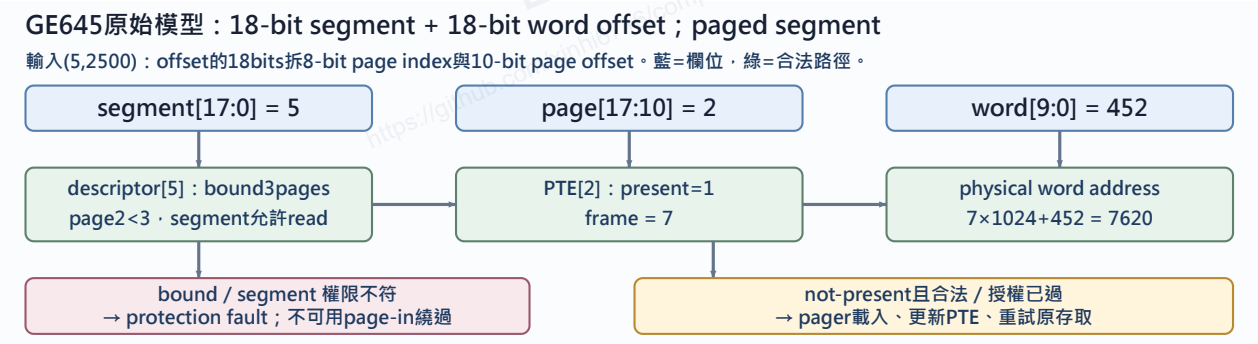
完整用完 level 0 至 k 的時間是 2^(k+1)−1。**成本模型**假設每次 dispatch 均支付 0.5，包括首次載入，也包括同一工作連續執行的 dispatch；它不主張真實 CTSS 每次都必然換出。四次成本合計 2，總時間 13；固定 RR 的 q=1 需 11 次 dispatch，成本 5.5，總時間 16.5。比較僅適用此長 burst 與成本假設。

Multics：以持續服務與選擇性共享組織系統

Multics 的 computer utility 結合階層命名、segment、paging 與受控共享。GE-645 的 virtual address 包含 18-bit segment number 與 18-bit word offset；page 大小為 1024 words，故 offset 可拆成 8-bit page index 與 10-bit page offset。GE-645 原始設計

追蹤 segment 5、word offset 2500：2500=2×1024+452，因此 page index=2、page offset=452。**教學狀態**假設 descriptor 已有效，段界限容許 page 0–2，且允許 read；PTE 2 已駐留且指向 frame 7。逐步檢查 segment→界限／讀取權限→PTE 2→frame 7，得到實體 word 位址 7×1024+452=7620。

若 segment descriptor 無效，先處理 segment fault；segment 合法但超出界限或未獲授權時，存取被拒絕，不能靠載入頁面繞過；合法且授權的 page 未駐留時，才可能交 pager 載入並重試。處理方式依系統狀態而定，並非每個 fault 都可恢復。



rings與分層不是同一種保證

Protection rings 約束跨權限層的存取與入口；THE layers 描述軟體抽象及依賴次序。軟體模組不必各有自己的 ring，畫出層次也不足以證明存取控制正確。

硬體世代要分清楚：GE-645 提供 segment access mode 等保護機制，但 ring 跨層呼叫與返回需 trap 至 supervisor，由軟體介入；後續 Honeywell 6180 的設計才將大部分 ring 檢查、跨層呼叫與返回移入硬體。詳見 Schroeder 與 Saltzer 原始論文。

能解析 segment 名稱不等於有權寫入；仍須通過存取權限與入口規則。共享 code、私人可寫狀態也必須由映射和保護規則維持。Multics 開發者回顧記錄實作整合後依實測成本反覆調整設計。

OS/360 family：相容目標與memory配置版本分開

IBM System/360家族希望不同容量機器可使用較一致的程式／工具；OS/360仍存在不同memory配置方案。MFT使用固定regions，MVT使用variable regions，不能把一個版本的配置行為當所有家族唯一規格。

固定region例可用RAM256KiB分4×64KiB，jobs50/50/70：兩個50各占64，共內部碎片28KiB；70即使總剩128仍不能放進單一64region。這是本例固定region限制，不是說OS/360不能有其他region配置。

variable allocation例holes [50,30,40]KiB，總120卻無連續70KiB，出現external fragmentation。移動／compaction是否可行還需relocation支援與成本；load-time binding的地址不能想改就改。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

virtual memory版本與CP/CMS的不同抽象

System/370之後的OS/VS及MVS利用新增virtual-memory硬體，與原始360沒有相同paging契約。CP/67提供virtual machine環境、CMS是其上可運作的single-user系統；「多個CMS」的隔離由CP的虛擬機資源管理支撐，不是CMS自行變成多使用者kernel。

給兩guest都見virtual address1000，經guest的memory視圖再由VMM映射到不同host資源，便能各執行自己的systemcode。guest的特權操作須由VMM中介，不能因guest認為自己supervisor就可改host所有memory。

family compatibility、硬體relocation、VM隔離與使用者互動是四個不同維度。JCL提供batch資源／步驟描述，不從選項數量直接推論correctness或某個固定效能比例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TENEX／TOPS-20把互動命令及memory支援一起設計

依原書脈絡，BBN在PDP-10相關平台加入paging支援並開發TENEX；DEC後續取得權利，形成DECSYSTEM-20／TOPS-20。command interpreter的互動輔助與virtual memory都是其案例重點，兩者作用不同。

命令completion／help降低使用者記憶完整語法的負擔；paging處理程式位址與駐留。shell能提示參數不代表它有權直接變更page table，pager也不能從memory保護替使用者猜出想下哪個命令。

將「像獨占自己的machine」的執行視圖與「多人同時使用」並列看，仍需CPU排程、terminal I/O、file權限與回收。互動性不能只由processor更快推出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

機制與可用性是互補的兩層

本例命令有`COPY source destination`，解析器辨識合法語法後只產生request；執行還需查來源read與目標parent／file的write契約。語法正確但無權限，應回授權錯誤，而不是以autocomplete成功當已准許。

若shell先檢查destination再隔一段時間以pathname寫入，名稱仍可能變，會遇第11章的TOCTOU。受管理的file reference、明確錯誤與可理解的互動回饋必須配合。

本例只是把介面／授權分工具體化，不宣稱TOPS-20逐字使用此toy命令語法；歷史UI特色與一般安全推論分開。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CP/M與MS-DOS：小型資源限制不等於完整隔離

CP/M在早期微電腦提供command與程式／裝置服務；MS-DOS在IBM PC生態延續單使用者／相容性路徑。原書強調其保護能力與後來多使用者受保護系統不同，不能把它們直接當縮小的現代NT或Linux。

64KiB、640KiB等描述需要區分CPU addressability、conventional memory layout、OS保留與擴充方案；不把一個常見program區間當成所有版本／硬體絕對physical RAM上限。

缺少受強制的memory隔離時，程式可誤寫system資料或其他內容。即使API規定應透過服務呼叫，也不能假設惡意程式一定遵守；需硬體／特權及受信任軟體共同中介。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單一address-space模型的越界反例

固定教學RAM [0,1024)，monitor占[0,256)，job占[256,1024)，沒有硬體write範圍檢查。job store address100可覆寫monitor，即使正確job的library從不這樣做。這是界限模型，不是MS-DOS真實位址配置。

加入受保護的base=256、limit=768且每次user位址a先驗0≤a<768，再轉physical256+a，便限制user存取自己的區間。a=−1及768都拒，a=0與767對應256與1023；一般user也不能改base/limit。

此保護不保證job不死loop，還需timer；也不解共享file政策。補上一個hardware feature只建立一類邊界，不會一次得到完整現代OS。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Macintosh與Windows：GUI不能決定核心保證

早期Macintosh與Windows使桌面互動成為重要介面；後續桌面平台逐步具有更完整memory、multitasking、networking等支援。比較時必須指定世代；本課Windows7為NT家族案例，不把早期依附DOS的Windows與其kernel契約混用。

GUI的windows、mouse、menu是使用者互動模型；preemption、memory protection、API相容與driver模型是另外的實作／隔離問題。畫出兩個視窗不能證明兩個process可真正平行、也不能證明彼此有受保護address space。

同樣，GUI卡住可能因UI thread阻塞，而非全部kernel不再排程。反應時間需追到事件、queue、UI handler及所等服務。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

cooperative與preemptive的可觀察差別

固定單CPU、工作A執行無限loop且從不yield，B已ready。cooperative模型只在A主動yield／阻塞時選B，因此B可永遠不跑；preemptive模型若timer可被服務且scheduler輪流分配，A最遲在slice邊界被搶占，B才有CPU。

若A其實在kernel持鎖不可搶占且無界loop，光有user slice也不能保證B進度；界限仍需包括kernel與interrupt延遲。這個toy反例比較兩種協定，不將每一代桌面OS都粗分成同一種。

對互動式系統，UI設計與scheduler的界限都重要；若某task持有全域資源，得到CPU的B仍可能等待它。因此「preemptive」不等於沒有deadlock或priority inversion。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mach版本與task／thread／port角色

Mach源於Accent相關研究，將task/address-space、thread、IPC與virtual memory作為核心抽象。task持address space與port rights等資源，thread是可排程執行context；多thread可在同task共享memory。

版本／模型	BSD功能放置	能推得的邊界
Mach2.x／2.5脈絡	大量BSD功能仍在kernel	不能說全部file服務都跨user server
Mach3 microkernel構想	UNIX功能移至user server	透過受控IPC跨address space

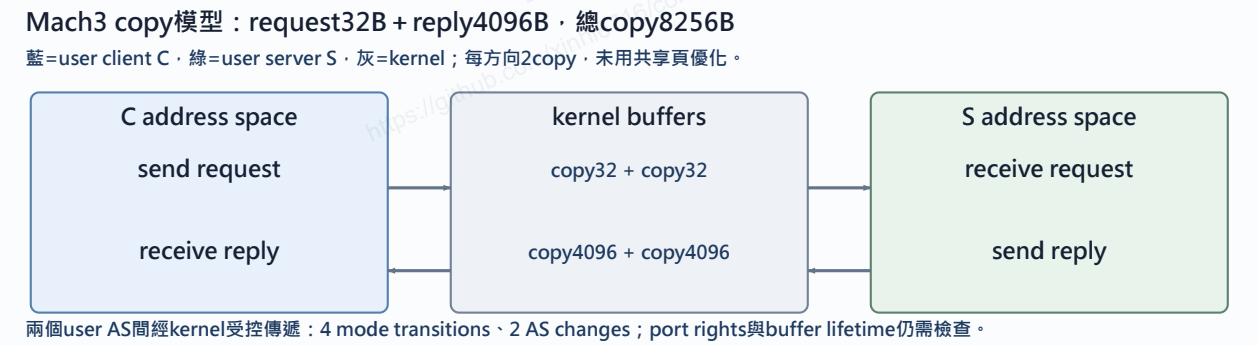
port right是可被kernel檢查的訊息authority；同名整數不是跨task自動有效權利。Mach亦支援shared memory，不能把「訊息是重要介面」擴寫成所有資料交換都必逐byte copy message。

同一request的IPC代價與隔離邊界

固定copy-based模型：client C送32B request給user server S，S回4096B；每方向都user→kernel buffer→另一user兩次copy，不使用shared-page／out-of-line優化。

路徑	mode transitions	address-space changes	copy bytes
C user→K→S user	2	C→S一次	32×2=64
S user→K→C user	2	S→C一次	4096×2=8192
合計	4	2	8256

server crash可被address-space邊界限制直接記憶體損害，但client仍需timeout／重建／錯誤處理；更小kernel不自動證明全部服務可用。訊息VM整合可減少某些copy，代價則轉到mapping、rights、page lifetime與COW，不等於零成本。



其他系統：不要把不同角色的處理器都當對等SMP

原書補充Burroughs MCP的系統程式語言、segmentation及多處理支援，以及CDC6600相關系統的多個處理活動／協調。這些例子提醒：processor角色、記憶體權限與I/O責任要先分清，不能只看「有多顆」就套相同scheduler模型。

高階語言可改善表達、型別／模組及維護，但仍需正確的同步、ABI與硬體控制。反過來，以assembly寫並不邏輯上推出錯誤；語言選擇改變可表達與可檢查的條件，不代替系統契約。

歷史系統是否延續還受相容軟體、硬體成本與部署組織影響。本章沒有從單一設計特色推論商業成敗，也不把書寫當時的市場敘述當成現況。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨案例機制對照

問題	案例	不可省略的條件
CPU與慢I/O重疊	resident monitor／spool	queue、buffer、裝置所有權及真正可重疊工作
資訊不全駐core	Atlas	mapping、fault重啟與完整victim選擇
非連續放置／共享code	XDS-940	relocation、唯讀code與private可變狀態
分解正確性推理	THE	無循環的依賴與逐層契約
將政策移出nucleus	RC4000	message ownership、pool上限及等待關係
互動／長burst取捨	CTSS	feedback／promotion／swap成本明列
選擇性共享與命名	Multics	segment、page與protection區分
跨服務隔離	Mach3模型	IPC authority、address space與failure處理

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從歷史演示回到一般證明

- Atlas三規則對非空frames必可選victim：有①用①；否則若有t≠0可在有限集合取最大score；其餘全t=0以③選T最大。這保證定義完整，不保證fault最少。
- buffer守恆由初始狀態與每次原子轉移維持；若buffer同時出現在free及reply，表示ownership轉移破壞不變量。
- layer歸納依Ci只依賴較低層且保留其契約；有cycle或不受控共享修改時，不能沿用。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz et al., Operating System Concepts, 9th ed., Ch. 20. Atlas one-level store; THE system; Multics GE-645; Multics protection.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study